

投影、连接、聚集、排序、索引等操作均已在 GPU 上实现,在 CPU-GPU 混合平台上创建的代价模型辅助数据库在两种平台之间分配不同的关系操作。GPU 数据库将 GPU 作为数据库的加速引擎,为数据库提供一些计算密集型操作的计算加速。随着 GPU 内存容量和内存带宽性能的不断增长,基于 GPU 内存的数据库技术可以提供更高的查询处理性能,在高实时性分析处理领域发挥越来越大的作用。GPU 数据库已成为高性能数据库的代表性技术,尤其在大数据实时分析处理领域得到广泛的关注。

17.2.3 新型非易失性内存

典型的计算机存储层次由内存(以动态随机存取存储器 DRAM 为代表的易失性存储器)和存储器(包括磁盘和固态硬盘)组成,内存支持字节级访问粒度,而存储器基于较大的数据块访问。

基于 3D XPoint 技术的 Intel Optane DC 是一种非易失性存储技术,它支持内存与存储器两种类型,一种是非易失性内存 Optane DC PM,另一种是基于 Optane DC 技术的固态硬盘(SSD)。

Optane DC PM 与 DRAM 相比容量更大,但读/写性能低于 DRAM,在应用中既可以作为大容量内存使用,也可以作为 DRAM 之外独立的非易失性存储设备使用。Optane DC PM 改变了传统存储层次的角色,DRAM 缓存热数据,而 Optane DC PM 作为持久存储的温数据层,提供接近 DRAM 性能的持久性内存存储访问能力,也弥补了内存数据库在 ACID 特性中对持久性 D 支持能力的缺失。

基于 Optane DC 技术的 SSD 与 SSD、磁盘一起构成新的存储层。非易失性内存的大容量和相对于 DRAM 的低成本也进一步降低了内存数据库的存储成本,推动内存数据库成为大数据、低成本、高性能的数据管理和分析处理平台。

新型非易失性内存的出现,改变了传统内存易失性的基本假设,为实现以内存为中心的统一存储结构与存储引擎提供了可能,对内存数据库技术的发展有着重要的推动作用。

随着 Optane DC PM 的推出,数据库厂商已开始研发基于非易失性内存的数据库实现技术。但非易失性内存相对于传统 DRAM 在读/写性能上的差距以及 DRAM-非易失性内存混合存储架构,还需要通过定制化的优化技术才能更好地发挥其性能。另一方面,非易失性内存存在产品化方面存在一定的不确定性,但对内存数据库而言,非易失性内存是提高内存数据库性价比的重要技术支持。

17.3 内存数据库的若干关键实现技术

通用的内存数据库管理系统要为用户提供 SQL 接口,具有内存存储管理、面向内存的查询处理和优化等基本模块,还应提供传统磁盘数据库管理系统的功能,如多用户的并发控制、事务管理和访问控制,能够保证数据库的完整性和安全性,在内存数据库出现故障时能够对系统进行恢复,等等。

内存数据库的性能受硬件特性影响较大,需要研究基于新的硬件体系结构下的实现技术。

下面简要介绍内存数据库中的若干关键技术。

17.3.1 数据存储

内存数据库将全部数据和临时工作数据集驻留在内存，采用面向内存优化访问的存储结构，以优化 CPU 访问和高速缓存(Cache)。

图 17.2 和图 17.3 显示了一个关系表 ORDER 使用磁盘行存储(page-slot)结构和内存列存储结构的示意图。

ORDERKEY ORDERSTATUS TOTALPRICE ORDERDATE

1	O	173665.47	1996/1/2
2	O	46929.18	1996/12/1
3	F	193846.25	1993/10/14
4	O	32151.7	1995/10/11
5	F	144659.2	1994/7/30
6	F	58749.59	1992/2/21
7	O	252004.18	1996/1/10
32	O	208660.75	1995/7/16
33	F	163243.98	1993/10/27
34	O	58949.67	1998/7/21

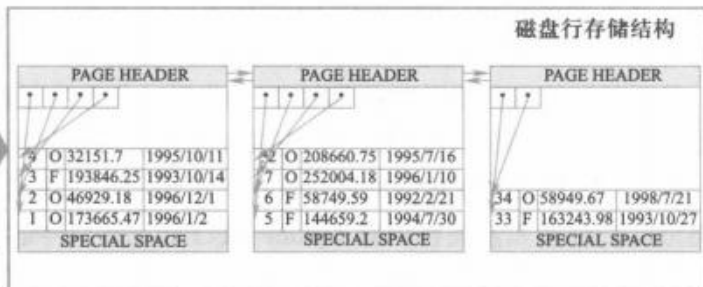


图 17.2 磁盘行存储结构

ORDERKEY ORDERSTATUS TOTALPRICE ORDERDATE

1	O	173665.47	1996/1/2
2	O	46929.18	1996/12/1
3	F	193846.25	1993/10/14
4	O	32151.7	1995/10/11
5	F	144659.2	1994/7/30
6	F	58749.59	1992/2/21
7	O	252004.18	1996/1/10
32	O	208660.75	1995/7/16
33	F	163243.98	1993/10/27
34	O	58949.67	1998/7/21

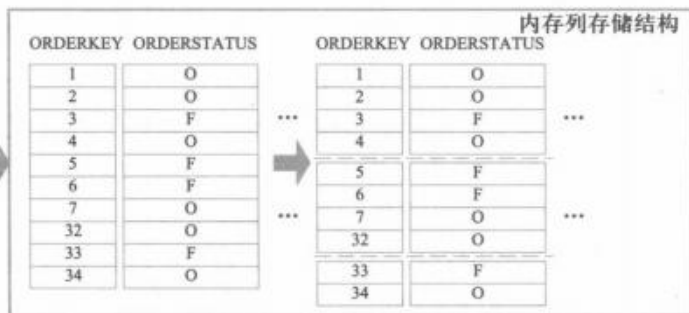


图 17.3 内存列存储结构

在图 17.2 中，磁盘数据库采用适应磁盘 I/O 访问的 page-slot 结构，记录以 slot 结构存储在磁盘页面中，记录通过块号和块内偏移地址进行定位，页面之间创建链表结构。数据库访问时将磁盘页面调入内存缓冲区，记录通过块号和偏移地址来进行查找。

在图 17.3 中，ORDER 表中的数据按列存储，记录可以通过内存指针或列偏移地址直接访问。内存列存储结构适用于数据仓库应用中的分析处理。

进一步地，内存列存储表中的记录可以按一定规则划分为行组，行组内采用列存储，行组内的列可以独立进行压缩。当记录存储为独立的列时，在记录增加时需要重新分配内存空间以容纳新的记录，采用行组结构则可以通过动态增加的行组结构存储新增的记录。

在内存-磁盘(或 SSD)硬件结构中，内存数据库的表、索引等数据结构在内存中采用适合内存访问的结构存储，但还需要存储在磁盘或 SSD 等外部设备上以获得持久数据存储能力，当系统重启时，数据需要从磁盘或 SSD 加载为内存数据结构。

17.3.2 查询处理及优化

内存数据库消除了传统数据库的磁盘 I/O 代价，其代价主要是 CPU 执行代价。CPU 执行代价主要体现在代码执行效率和 Cache 访问效率两个方面。行处理模型、列处理模型、向量化查询处理模型和实时编译查询处理模型是其中比较有代表性的几种查询处理技术，分别从内存访问效率、代码执行效率、Cache 访问效率等方面优化查询处理性能。

传统的查询处理模型是基于行存储结构的迭代处理模型，查询引擎基于语法树迭代解析每条记录以完成查询处理过程。图 17.4 显示了查询 $\text{Price} * \text{Quantity} * (1 - \text{Tax})$ 表达式的不同查询处理模型。

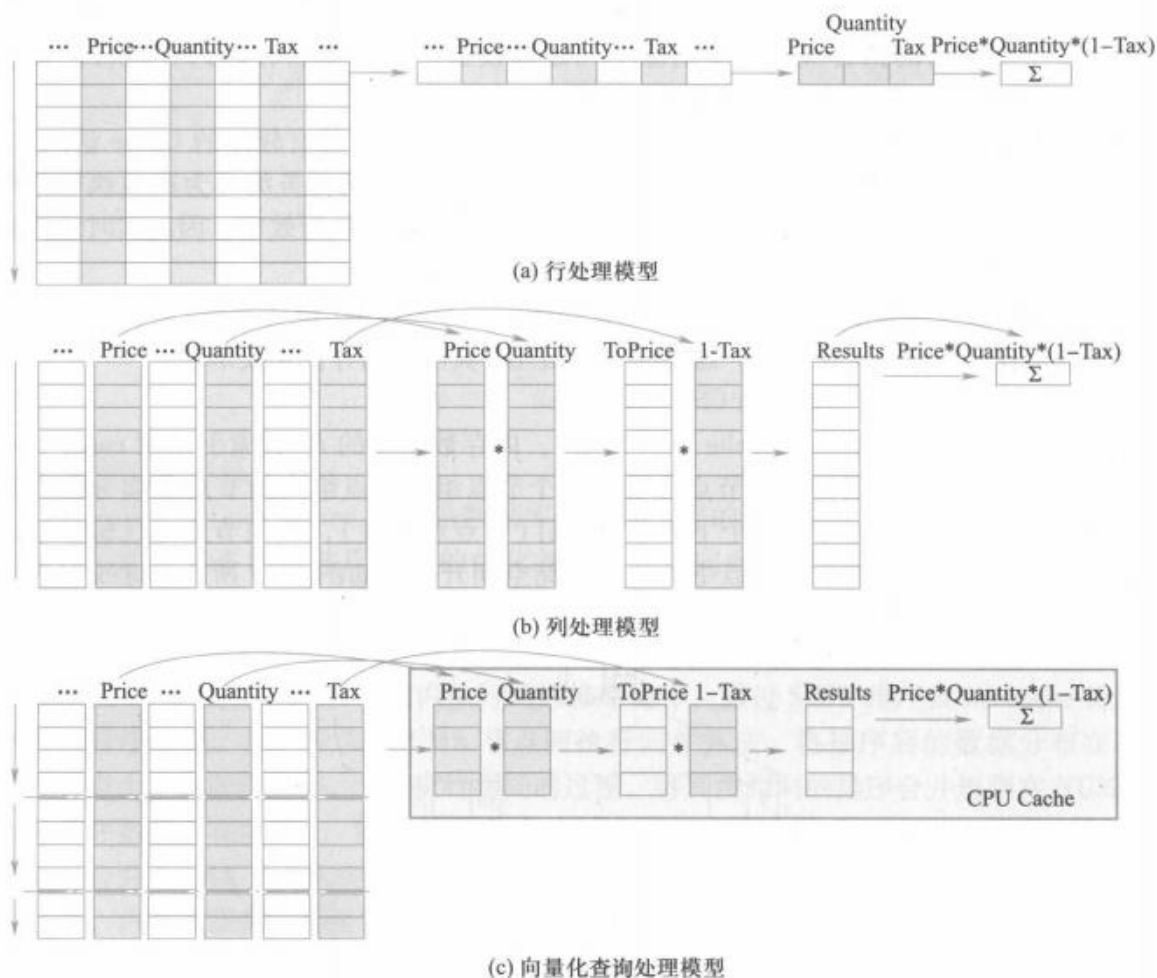


图 17.4 查询处理模型示例

如图 17.4(a) 所示，行处理模型需要查询引擎迭代访问每一条记录，为每一条记录查询关系的元数据，解析查询属性偏移地址并访问查询相关的属性，计算并累加查询表达式。在行处

理模型中,重复执行的记录迭代解析及相应的函数调用执行代价较高。

如图 17.4(b)所示,列处理模型^[10]一次解析并处理整个列,如执行列 Price 和列 Quantity 中所有记录的乘积操作,将中间结果记录在 ToPrice 临时列中。然后再将 ToPrice 列与 Tax 列执行 $ToPrice * (1 - Tax)$ 计算,将列计算结果存储在中间结果列 Results 列中,最后通过对 Results 列中记录的累加计算出查询结果。列处理模型一次解析和函数调用处理全部的列数据,执行代价被大量记录分摊。但一次一列的处理模式需要较大的临时列暂存中间结果数据,产生较大的内存空间消耗和访问代价。

如图 17.4(c)所示,向量化查询处理模型基于 Cache 大小将列数据水平划分为适当大小的向量(如 1 024 个数据作为一个向量),查询引擎以向量为单位执行查询处理任务,记录的解析和函数调用代价被向量中的数据分摊,而且向量化的中间结果列存储于低延迟的 Cache 中,既具有列处理模型较高的执行效率,又消除了列处理模型中间结果暂存的内存代价,实现了 Cache 内的查询处理,提高了查询处理性能。

向量化查询处理模型是当前内存数据库的代表性优化技术,具有较好的 Cache 访问效率。随着编译器技术的发展,现代数据库应用实时编译技术^[11]将查询任务编译为高效执行的机器码或中间级字节码,最小化记录解析与函数调用代价,提高代码执行效率。因此实时编译技术也是现代内存数据库代表性的优化技术。

内存数据库在底层算法实现方面大多采用 hardware-conscious(硬件敏感)的算法设计,基于硬件特性来设计算法与优化技术。这里简单介绍几种典型的内存优化技术。

1. Cache-conscious B+树索引(CSB+树)

内存数据访问的最小单位是 cache line(64 B),内存数据库的 B+树索引采用 cache line 大小的节点存储键值,同一层的索引节点设置为一个节点组,节点组内的节点连续分配内存地址,从而使父节点可以只保存第一个下级节点指针(节点组指针),其他节点通过偏移地址计算获得其内存地址,以节省索引节点中的地址存储空间开销。如图 17.5 所示,虚线框内为节点组,箭头为指向第一个节点的地址指针。

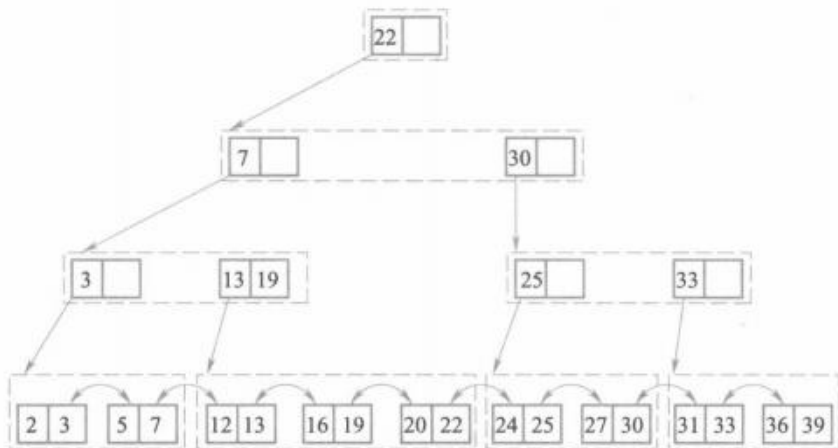


图 17.5 CSB+树示例

2. Radix 分区哈希连接

这种方法通过 Radix 分区技术减少 R 表和 S 表在分区过程中所产生的 Cache 失效和 TLB 失效。如图 17.6 所示, 在分区阶段, 采用 TLB 缓存优化的多趟分区方法将 R 表和 S 表按相同的 Radix 分区算法划分为较小的分区; 在构建哈希表阶段, 各分区上独立创建适合 Cache 大小的哈希表; 在哈希探测阶段, S 表分区探测 R 表分区哈希表, 完成连接操作。基于分区的哈希连接操作在分区阶段需要较大的空间和时间代价, 但分区后的哈希连接能够满足哈希表小于 Cache 容量的要求, 使得在哈希探测阶段获得较好的性能。

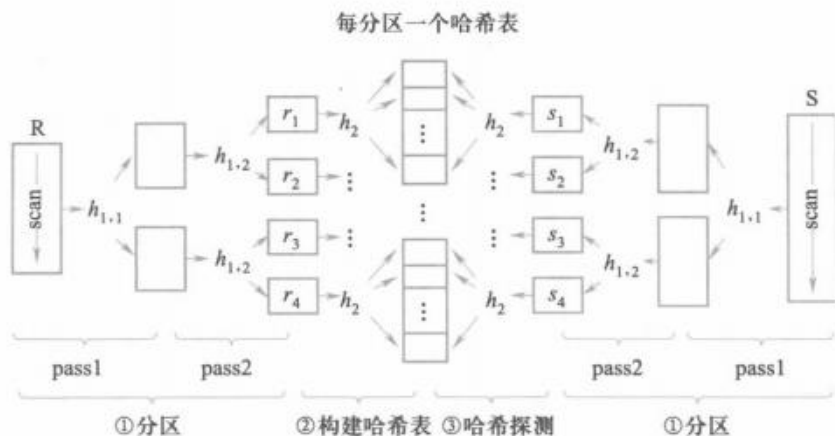


图 17.6 Radix 分区哈希连接

3. 排序合并连接

排序合并连接(sort-merge join)是数据库的一种主要的连接算法。内存数据库的排序合并算法设计需要考虑不同硬件层面上的优化技术, 如面向向量寄存器的 SIMD 排序算法, in-cache 排序算法等。图 17.7 所示的 NUMA-aware 排序合并连接算法^[12]过程优化了基于不同 NUMA 节点内存之间的排序合并连接算法。

在阶段 1, R 表在 NUMA 节点内执行局部排序操作, 通过 SIMD 排序及 in-cache 排序优化本地节点排序性能。在阶段 2, NUMA 节点间执行归并算法, 将排序后的数据分布在不同的 NUMA 节点上。在阶段 3~4, S 表执行相同的过程, 将其按相同的排序合并规则在 NUMA 节点中划分排序数据。在阶段 5, 各 NUMA 节点执行本地排序后 R 表和 S 表分区上的归并连接操作, 最后合并 NUMA 节点间的连接结果。

总之, 内存数据库在算法实现和优化技术上面向现代多核处理器的硬件特性做了深入的优化工作, 如面向 SIMD 的算法实现、面向多线程的并行优化技术、面向多级 Cache 的优化技术等, 以最大化发挥硬件效率, 提高查询处理性能。

随着 GPU、FPGA、Phi 等硬件加速器技术的成熟与应用的普及, 面向 GPU^[13]、FPGA^[14]、Phi^[15] 的内存数据库硬件加速技术成为新的发展趋势。硬件加速器更为强大的并行计算能力, 为内存数据库提供了更高性能的查询加速引擎。

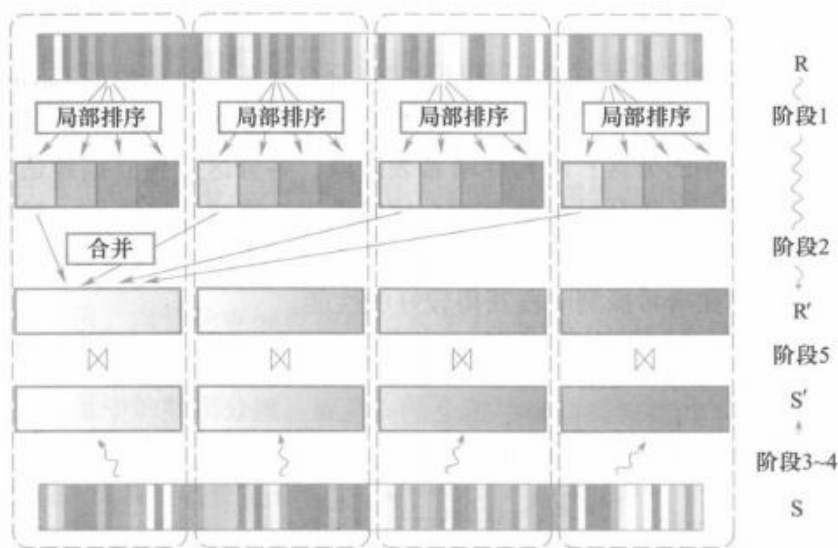


图 17.7 NUMA-aware 排序合并连接算法过程

17.3.3 并发与恢复

1. 并发控制

内存数据库与磁盘数据库的并发控制机制类似，但在细节上存在一定差异。由于数据存储在内存中，内存数据库中的事务执行时间一般较短，因此持锁时间也较短，系统中冲突较少，所以可以采用以下方法减少锁的开销：采用较大的封锁粒度（如表级锁）、采用乐观加锁方式、减少锁的类型、将锁信息存储在数据本身。

对于内存数据来说，封锁产生的 CPU 代价会对性能产生严重的影响，特别是对于工作负载主要由短小事务构成的联机事务处理应用场合，每个事务要求极短的响应时间，在几十毫秒甚至微秒之内完成。针对此问题，S. Blott 等提出了接近串行的并发控制协议。该协议的特点是：写事务在整个数据库上施加互斥锁（mutex），通过时间戳和互斥锁在事务的提交记录没有到达磁盘之前允许新事务开始，并且保证任何提交的读事务不会读到未提交的数据。无锁数据结构（lock-free data structure）也是内存数据库的一项优化技术，通过原子级操作实现无锁并发控制，提高内存数据库的并发查询处理性能。

并发控制会带来一些系统代价，如 CPU 代价、存储代价等，影响系统性能。而内存数据库对性能要求非常高，所以利用内存优势并结合应用需求，在保证事务 ACID 特性的同时尽量减少并发控制对性能的影响，是内存数据库需要进一步研究的问题。

并发查询处理需要解决的问题还包括不同查询任务并发执行时如何高效地解决访问冲突，如联机事务处理与联机分析处理负载混合执行时的并发访问控制问题。Hyper 提出了基于操作系统 snapshot 机制的联机事务处理和联机分析处理混合负载并发执行技术，通过操作系统级的 snapshot 隔离联机事务处理更新数据和联机分析处理只读查询数据，支持不同类型查询任务的

并发执行。

2. 恢复机制

由于 DRAM 内存的脆弱性和易失性，内存数据库中的数据容易被破坏和丢失，所以内存数据库数据需要在磁盘等非易失性存储介质中进行备份，并且在对数据更新时将日志写到非易失性存储介质中。

在内存数据库中，如果在事务提交时将日志写入磁盘，则由于写日志所产生的磁盘 I/O 会延长事务的处理时间，降低内存数据库的性能。所以，将日志写在何处以及何时将日志写入磁盘，在内存数据库中是一个非常重要的问题。一些研究者提出了预提交、组提交等方法来降低日志 I/O 的代价，并提出使用 PCM、flash memory 等非易失性内存存储日志的方法。首先将日志存储在非易失性内存中，然后提交事务，再异步地把日志写入磁盘。内存数据库的索引构建非常快，一些内存数据库并不记录索引更新的 redo 日志，而将 undo 日志保存在内存中，系统恢复时直接重建索引结构。一些内存数据库通过只记 redo 日志的方式降低日志代价，通过周期性的检查点技术保证未提交事务数据不写入磁盘，在系统恢复时加载检查点并重做 redo 日志。也可以通过记录的多版本机制消除记录的原地更新(in-place update)，未提交事务的记录版本最终被垃圾清理进程清除。

在发生系统崩溃时，如何从备份和日志中恢复数据也是内存数据库重要的性能指标。为了能够尽快地恢复系统的使用，一般可通过两步来恢复数据：首先恢复热点数据，即执行事务所必需的数据，然后在后台恢复其他非热点数据。另外，也可根据数据在磁盘上的存储顺序、优先级(是否为热点数据)以及访问频率等参数来确定数据的装载顺序。为了提高内存数据库的恢复性能，一些内存数据库采用了多核并行恢复技术，该技术需要将数据和日志进行分区，以保证分区的日志仅与分区的数据相关，每个核心可以并行地执行相应数据与日志上的恢复任务。

17.4 内存数据库的几种实现方案

针对不同的应用目标，内存数据库有不同的实现方案。本节介绍三种主要的方案：混合的内存加速引擎、独立的内存数据库系统和 GPU 数据库。

17.4.1 混合的内存加速引擎

这种方案是在传统数据库的磁盘处理引擎基础上，通过集成内存数据处理引擎技术来提升数据库的实时处理能力。当前的发展趋势是混合双/多引擎结构数据库，图 17.8 为几种产品示例。

图 17.8(a)为 Oracle 公司推出的支持两种存储格式的内存数据库产品 Oracle Database In-Memory，其行存储结构用于加速内存联机事务处理负载、列存储结构用于加速内存联机分析处理负载。列存储引擎是完全内存列存储结构，应用 SIMD、向量化处理、数据压缩、存储索